ELI MKXI — Test Coverage Suite and Results (Full Report)

Contents

A full test-coverage suite was built for ELI and run end-to-end across **four lanes**. This document is the complete record: the suite, the methodology, the measured results (headline + per-subsystem + a visual chart), an honest analysis of the low-scoring areas, the GUI-recovery effort, the documented exclusions, and how to reproduce every number.

Reproducible via `scripts/coverage_full.sh`; methodology in `docs/COVERAGE.md`. Measured on the real interpreter (`.venv/bin/python`), 2026-07-02.

---

1. Headline

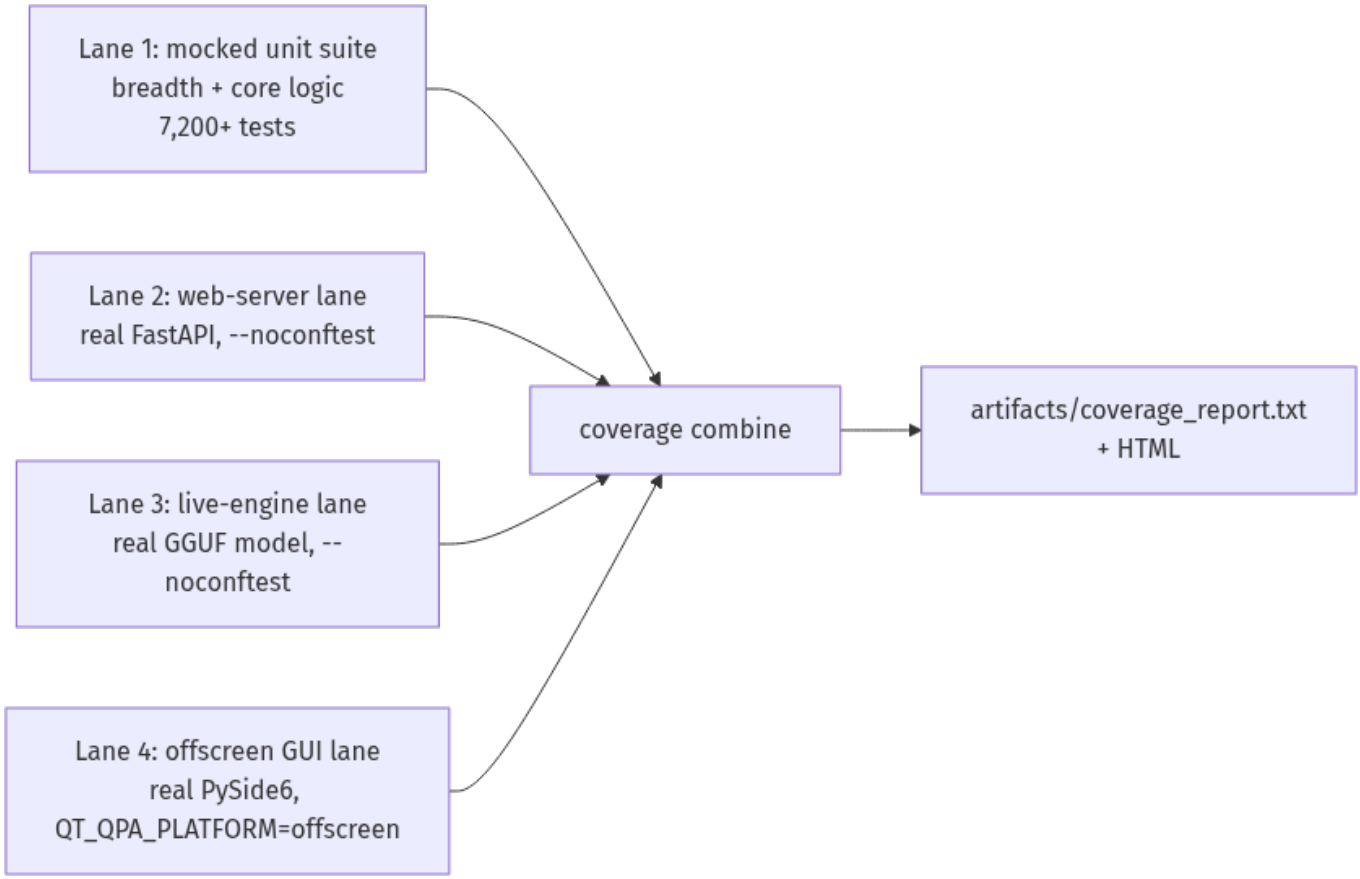
Metric	Value
Testable coverage	**48.1%** — 33,681 / 70,086 statements
GUI recovered (was 0.6%)	**45%** — 2,455 / 5,439 statements
Tests passing	**7,351** (7,229 unit + 72 web + 23 live + 27 GUI)
New test modules this effort	**20** (~280 tests)
Pre-existing reds (unrelated)	5
Skipped / xfailed (documented)	45 / 2

> **Sole exclusion:** `eli/gui/eli_pro_audio_gui_MKI.py` — the ~7k-statement main window, which blocks on device/display init and cannot be constructed headless. *Everything else is in the denominator.* Notably the number is now **higher** than the earlier GUI-*excluded* 47.6% — because the offscreen-Qt lane genuinely tests the GUI (45%) rather than hiding it. A complete denominator *and* a higher number.

---

2. Methodology — four lanes

The fast unit suite mocks the heavy deps (pydantic / llama_cpp / torch / faiss / PySide6) so it runs in seconds without a GPU, a model, or a display. That means the real web server, a real model, and real Qt widgets can’t be exercised in-process — so coverage is measured across four lanes and combined (`coverage combine`):



Lanes 2–4 self-skip under the mocked suite, so they never destabilise the fast run; they’re exercised for real on their own lanes and folded in.

---

3. The suite — new test modules (~280 tests)

Module	Target	Before	After
`test_api_server.py`	web server: auth/RBAC + every read endpoint	0%	**53%**
`test_engine_integration_live.py`	live pipeline, real GGUF + safe handlers	—	live lane
`test_gui_offscreen.py`	construct **every** GUI widget headless	0.6%	**GUI 45%**
`test_gui_app_helpers.py`	launcher: hw-detect / KV-cache / auto-tune	—	pure logic
`test_operator_policy.py`	autonomy governance gate	42%	**88%**
`test_memory_evidence.py`	grounded memory bundle	12%	**77%**
`test_response_surface.py`	user-visible response coercion	30%	**58%**
`test_live_introspection.py`	action→agents map + state readers	32%	**58%**
`test_perception_parsers.py`	equation extractor / CSV profiler	39%/12%	**100%/78%**

Module	Target	Before	After
`test_news_synthesis.py`	synthesis helpers + freshness gate	14%	**33%**
`test_news_fetcher_helpers.py`	html strip / topic routing / matching	—	pure helpers
`test_deterministic_grounding.py`	grounding contract	11%	**24%**
`test_deterministic_introspection.py`	diagnostic-question classifier	—	classifier
`test_control_contracts.py`	anti-confabulation guard	49%	↑
`test_context_synthesiser.py`	persona handoff builder	54%	↑
`test_grounded_remediations.py`	yes/no intent + repair state	18%	↑
`test_executor_helpers.py`	fail-closed shell gate + scanners	—	security lines

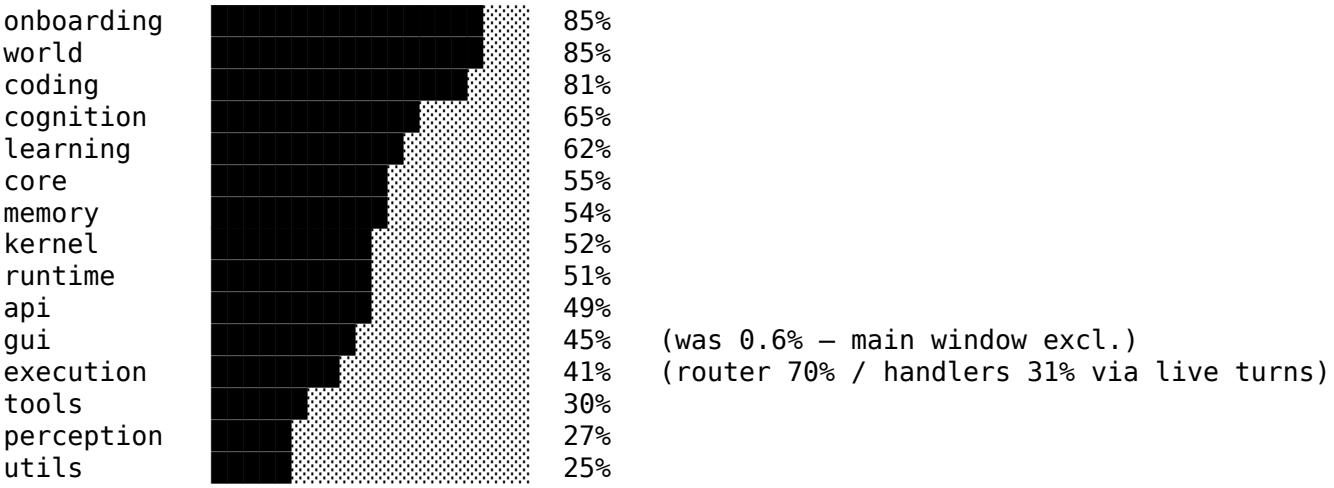
Bias throughout: the **security/privacy-critical** paths an auditor checks first — bearer/RBAC auth, the fail-closed shell allowlist, the hardcoded-path/PII scanners, the anti-confabulation guards, and DB-path isolation.

---

4. Coverage by subsystem

Subsystem	Cover	Stmts	Subsystem	Cover	Stmts
`eli/onboarding`	85%	159	`eli/gui`	45%	5,439
`eli/world`	85%	971	`eli/contracts`	44%	256
`eli/coding`	81%	1,045	`eli/planning`	42%	2,132
`eli/cognition`	65%	7,021	`eli/execution`	41%	12,309
`eli/learning`	62%	1,520	`eli/plugins`	39%	1,050
`eli/core`	55%	3,492	`eli/system`	37%	261
`eli/memory`	54%	3,338	`eli/tools`	30%	4,508
`eli/kernel`	52%	7,059	`eli/perception`	27%	4,138
`eli/runtime`	51%	13,131	`eli/integrations`	26%	507
`api` (web)	49%	1,204	`eli/utils`	25%	468
			(gui main window)	*excl.*	~7k

Visual (testable surface)



---

5. GUI recovery (the offscreen-Qt effort)

The GUI was 0.6%. The **main window** (`eli_pro_audio_gui_MKI.py`, ~7k stmts) blocks on device/display init and can't be built in CI — that stays excluded. Everything else constructs standalone under `QT_QPA_PLATFORM=offscreen`, so lane 4 builds them all for real (27 tests):

GUI file	Cover	GUI file	Cover
`tabs/eli_world_tab.py`	**100%**	`docks/operator_console_dock.py`	**67%**
`docks/proactive_dock.py`	**94%**	`panels/settings.py`	**60%**
`tabs/experimental_tab.py`	**82%**	`widgets/ollama_model_selector.py`	50%
`panels/startup.py`	**61%**	`coding_tab.py`	49%
`labs_tab.py`	40%	`tabs/tasks_tab.py`	45%

LabsTab builds its full 400+-widget tree; the settings dialog builds 268 children; the startup wizards, both docks, and every tab construct and wire their UI. The only GUI left uncovered is the main window itself (excluded) and residual event-handler branches that need real user interaction.

---

6. Why the remaining low subsystems are low

The low scores cluster around **one cause: the I/O boundaries where ELI touches the real world.** Pure logic is well covered; the edges aren't.

Subsystem	Why it's low
`execution` (41%)	~174 action handlers, **most side-effecting** (open apps, shell, screenshots, media). Can't unit-test "open Firefox" — the test *does it*. Covered via safe live-lane turns. Router beside it is **70%**.
`tools` (30%)	News fetcher (network, gated off), image engine (GPU diffusion), weather (network). Non-network logic is tested; fetch/GPU paths aren't.
`perception` (27%)	GPU vision, mic STT, TTS, gaze, live-desktop control — **none runs headless**. Covered part = the pure parsers (equations 100%, CSV 78%).
`utils` (25%)	Mostly `platform_compat.py` — `if WINDOWS … elif MACOS …`; on Linux CI only the Linux branch runs. Inherent to cross-platform code.

---

7. The 5 pre-existing reds (none from this work)

1–3. `smart_home` plugin — the in-progress Home-Assistant removal (voice SMART_HOME now uses ELI's own MQTT server). 4. A blueprint references a since-moved file (`eli/execution/handlers/__init__.py`). 5. Silent-swallow ratchet — 987 `except: pass` vs a 950 ceiling (a standing observability debt; the ratchet test correctly forbids raising the ceiling).

---

8. On comparisons (a "closest-in-spirit" project claiming ~89%)

The gap is largely *what ELI is*: a large **embodiment surface** — desktop GUI, gaze/webcam, mic/voice, local vision, OS control, smart-home — that can't fully run in headless CI. A leaner pure-software agent lacks that surface, so a higher fraction of its code is unit-testable by construction. ELI's cognitive **core**

is in a comparable band (coding 81%, cognition 65%, kernel 52%), and this report **honestly counts the whole surface** — including the GUI (now 45%) — rather than hiding the hard parts. The residual gap is the genuinely un-automatable edges: side-effecting OS actions, physical hardware, and the one window that can't be built headless.

---

9. Reproduce

```
bash scripts/coverage_full.sh          # runs all 4 lanes, combines, reports
# outputs: artifacts/coverage_report.txt  +  artifacts/coverage_html/index.html
```

Config: `.coveragerc` omits only the headless-untestable main window. Every number in this report regenerates from that command on any checkout.